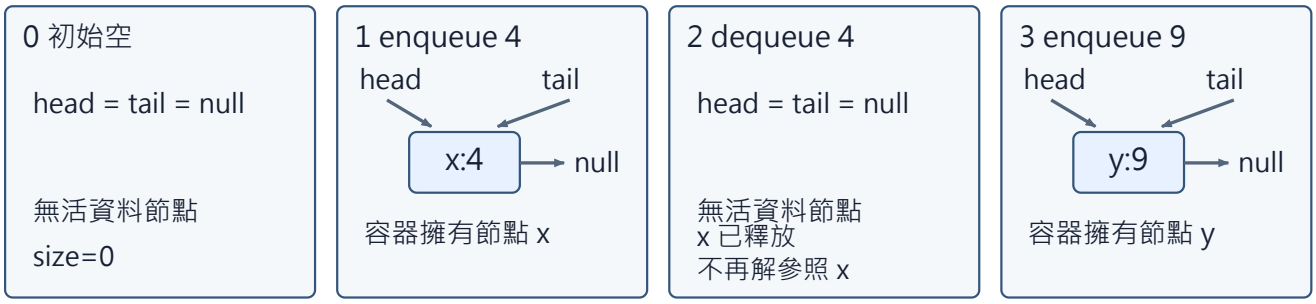
第 4 章推導解答

DS04-01 • 最後一個佇列節點

初始 head=tail=null。enqueue 4 建新節點 x，令 head=tail=x、x.next=null。dequeue 保存值並移除 x 後，head=null；因佇列已空，tail 也必須設 null，再釋放 x。enqueue 9 建 y，令 head=tail=y。若第二步 tail 仍指 x，下一次沿 tail.next 寫入會使用已釋放物件。這是所有權和生命週期違規，不只是顯示內容不正確。

移除最後一點：先清 head 與 tail，再釋放該節點



DS04-02 • 原地反轉證明

初始 prev 空、curr=3→8→11。第一輪 prev=3、curr=8→11；第二輪 prev=8→3、curr=11；第三輪 prev=11→8→3、curr 空。每輪先保存 curr.next，才反轉當前 link，故未處理後綴仍可達。兩段無環且節點互斥，聯集固定為原節點集合。終止後 head=prev，舊 head 即新 tail。空串列時兩者仍為 null，單點時 head 與 tail 指同一點。

DS04-03 • 多項式正規化與相加

P 有同指數 4 重複且未嚴格遞減，合併後 $P=x^4+x$。$Q=-x^4+5$，相加得到 $x+5$，四次項係數為 0 因而不保留節點。任意順序的 p 個輸入項若採比較排序，正規化為 O(p log p) 排序加 O(p) 合併。兩個已正規化串列相加才是 O(p+q)，不能把未排序原始輸入的成本一併忽略。

DS04-04 • 雙向串列刪除

先令 p=x.prev、q=x.next，再設定 p.next=q、q.prev=p，最後釋放 x 並減少 size。其餘 links 沒變，而改動的兩鄰點滿足 p.next.prev=p、q.prev.next=q，因此一致性恢復。前提為 x 是本容器活著的資料節點，非 sentinel，且沒有重複刪除。若外部可交入任意 pointer，邊界需有 owner 驗證或不接受該介面，不能假設任意指標都合法。

本段固定採兩個不含資料的 sentinel H、T。H.prev=null、T.next=null；空串列為 H.next=T、T.prev=H，size=0。每個資料節點的 prev、next 均非 null，因此以下 p.next 與 q.prev 可安全解參照。若改採無 sentinel 的 null 端點，刪首／尾節點必須另更新 head／tail，不能直接套此程式。

第 4 章推導解答 （續）

只有一筆資料時 H↔x↔T，刪除取 p=H、q=T，接回 H.next=T、T.prev=H，size 由 1 降 0；H、T 保留且絕不釋放。刪多筆串列的第一或最後資料同樣由鄰接 sentinel 接回。合法節點 handle 與所屬容器的確認在進入指標操作前完成。

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study